



Programación del Lado del Cliente

Unidad III

Programación del Lado del Cliente

3.1 Introducción al Lenguaje

3.2 Manejo de Frameworks

3.3 Estructuras de Control

3.4 Manipulación de Objetos

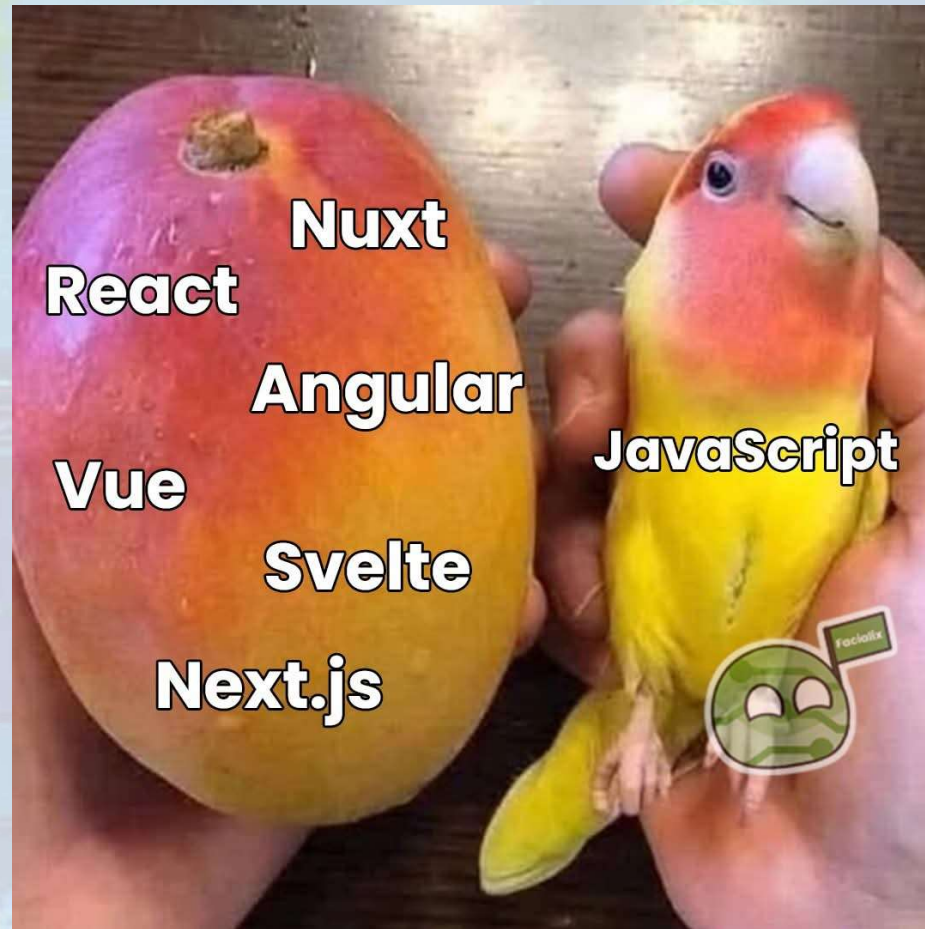
Programación del Lado del Cliente

When veo a alguien usando



¿No estaban extintas esas cosas?

Programación del Lado del Cliente



Programación del Lado del Cliente



Practica Unidad 3: Programación del Lado del Cliente

- Introducción:

- Servidores Web

- HTML

- CSS

- Validador Oficial

- HTML: <https://validator.w3.org/>

- CSS: <https://jigsaw.w3.org/css-validator/>

- GitHub

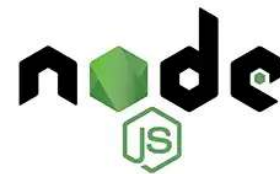
- JavaScript

- Librerías

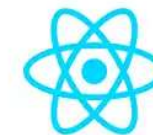
- Frameworks



NODE JS



REACT JS



ANGULAR

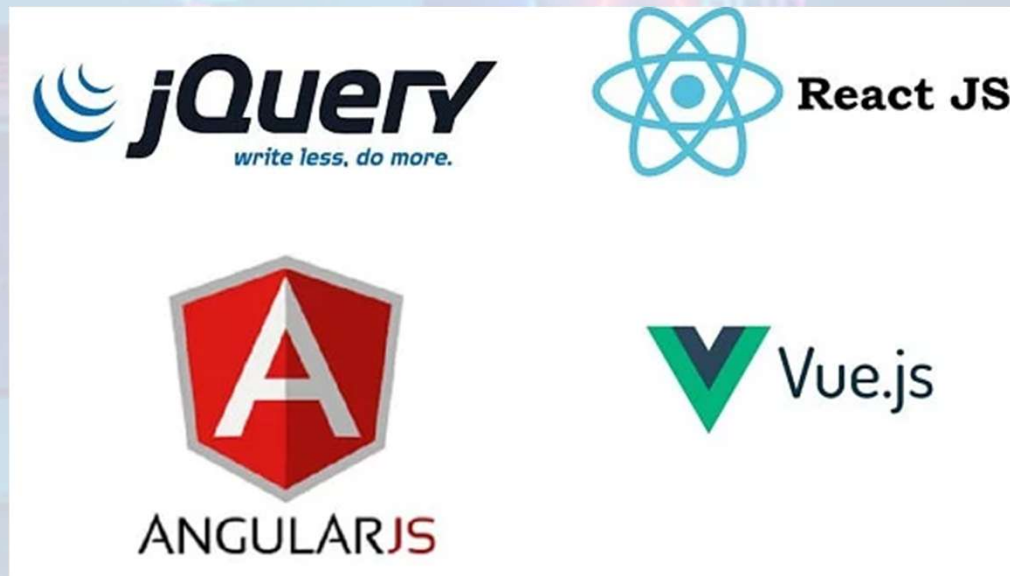


VUE JS



Practica Unidad 3: Programación del Lado del Cliente

- Objetivo:
 - Conocer y aplicar programación del lado del cliente con JavaScript para la construcción de aplicaciones Web dinámicas.
 - Implementar librerías y Frameworks



Practica Unidad 3: Programación del Lado del Cliente

- Material y Equipo:
 - Laptop o computadora de escritorio
 - Servidores Web
 - Validadores oficiales
 - GitHub
 - JavaScript
 - Librerías
 - Frameworks



Practica Unidad 3: Programación del Lado del Cliente

- Descripción de la Problemática:
 - Basado en la problemática anterior, la empresa DevWeb necesita agregar eventos y validaciones sobre las tablas, objetos y formularios a través de librerías y Frameworks conocidos.

VALIDACIÓN DE FORMULARIOS

Usuario Dostin123  El usuario tiene que ser de 4 a 16 dígitos y solo puede contener números, letras y guion bajo	Nombre Dostin Hurtado  ¡Excelente!
Contraseña ••••••••  ¡Excelente!	Repetir Contraseña ••••••••  Ambas contraseñas deben ser iguales

Practica Unidad 3: Programación del Lado del Cliente

- Paso 1:
 - Agregar eventos y elementos dinámicos a los objetos del índice utilizando al menos una librería o un Framework conocido.
 - Considerar estos eventos y elementos para el resto de las paginas para cuando se muestre todo el contenido de la BD.
- Paso 2:
 - Aplicar validacion del lado del cliente para todos los formularios de la pagina.
 - Utilizar al menos una libreria o un framework conocido.
- Paso 3:
 - Aplicar el manejo de versiones y actualizaciones del codigo fuente en GitHub.
 - Realizar cambios en el repositorio remoto del compañero.
- Paso 4:
 - Seguir validando el codigo fuente HTML de todas las paginas renderizadas.
 - Seguir validando los CSSs.

Practica Unidad 3: Programación del Lado del Cliente

- Entregables:
 - PDF con la siguiente información:
 - Incluir en orden los 4 pasos de la practica
 - Breve descripción del procedimiento
 - Comandos y/o configuración realizada
 - Captura de pantalla completa
 - Conclusiones
 - Código Fuente
 - Revisión
 - Opción 1: presentación frente al grupo en la sala 0
 - Opción 2: realizar un video de 10 minutos narrando los 4 pasos de la practica (casos justificables)

Practica Unidad 3: Programación del Lado del Cliente

- Evaluación:
 - En parejas
 - Derecho a Evaluación:
 - Tutorial JavaScript: 200 lecciones, 267 ejercicios, 1 examen
 - Tutorial con todas las lecciones, ejercicios y examen de solo una librería (jQuery, React) o framework (Vue, Angular)
 - Tutorial FreeCodeCamp: Front-End Development Libraries Certification
 - Entregar al menos una de las 3 actividades de la Unidad
- Fecha de entrega:
 - 23 al 25 de marzo: 100
 - 26 y 27 de marzo: 80
- Por cada respuesta o elemento faltante se restaran 10 puntos
- Puntos extra por usar elementos diferentes a los ejemplos de clase
- Todos los archivos (PDF, video) deben tener el nombre con la siguiente estructura:
 - *AP1#CONTROL_AP2#CONTROL_U3_PW_G.EXTENSION*

3.1 Introducción al Lenguaje

- JavaScript es el lenguaje de programación de la web.
 - Puede calcular, manipular y validar datos.
 - Puede actualizar y cambiar tanto HTML como CSS.
- JavaScript es uno de los tres lenguajes que todos los desarrolladores web deben aprender:
 1. HTML para definir el contenido de las páginas web
 2. CSS para especificar el diseño de las páginas web
 3. JavaScript para programar el comportamiento de las páginas web

Qué es
Javascript

```
<script type="text/javascript">
switch (new Date().getDay()) {
  case 6:
    text = "Friday";
    break;
  case 0:
    text = "Sunday";
    break;
  default:
    text = "Choose Your Day";
  }
</script>
```

3.1 Introducción al Lenguaje

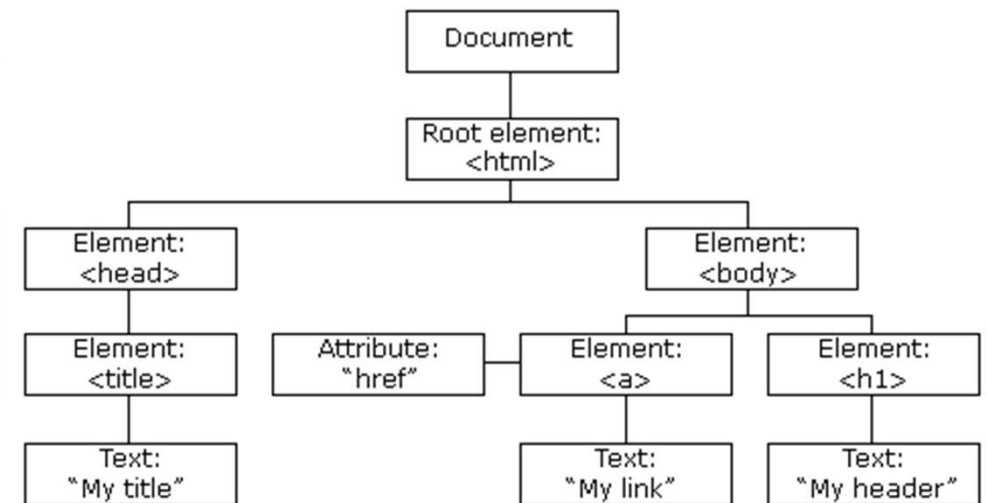
- JavaScript , inventado por Brendan Eich en 1995, se convirtió en un estándar ECMA en 1997.
- ECMAScript es el nombre oficial del estándar JavaScript .
- A partir de 1997, las versiones de ECMA se abreviaron con números (ES1, ES2, ES3, ES5, ES6).
- 2016 - 2025 , las versiones se nombran por año (ECMAScript 2016, 2017, 2018 ... 2025).

Official Name	Most Important Features
ECMAScript 1 (1997)	First edition
ECMAScript 2 (1998)	Only editorial changes
ECMAScript 3 (1999)	<u>Regular expressions</u> <u>try ... catch</u> <u>switch</u> <u>do-while</u>
ECMAScript 4	Never released
ECMAScript 5 (2009)	<u>use strict</u> <u>JSON Support</u> <u>String.trim()</u> Array.isArray() Array iteration methods Allows trailing commas for object literals
ECMAScript 6 (2015)	Added let and const Default parameter values Array.find() Array.findIndex()

3.1 Introducción al Lenguaje

- El DOM (Document Object Model) HTML
- Cuando se carga una página web, el navegador crea un **Document Object Model** de la página.
- El modelo **DOM HTML** se construye como un árbol de **objetos** :
 - JavaScript puede cambiar todos los elementos HTML de la página
 - JavaScript puede cambiar todos los atributos HTML de la página
 - JavaScript puede cambiar todos los estilos CSS de la página
 - JavaScript puede eliminar elementos y atributos HTML existentes
 - JavaScript puede agregar nuevos elementos y atributos HTML
 - JavaScript puede reaccionar a todos los eventos HTML existentes en la página
 - JavaScript puede crear nuevos eventos HTML en la página
- Interface de programación
 - Los elementos HTML como **objetos**
 - Las **propiedades** de todos los elementos HTML
 - Los **métodos** para acceder a todos los elementos HTML
 - Los **eventos** para todos los elementos HTML

The HTML DOM Tree of Objects



3.1 Introducción al Lenguaje

- En HTML, el código JavaScript se inserta entre las etiquetas `<script>` y `</script>`
- Los scripts se pueden colocar en la sección `<body>`, o `<head>` de una página HTML, o en ambas.
- Los scripts también se pueden colocar en archivos externos
 - Separa HTML y código
 - Hace que HTML y JavaScript sean más fáciles de leer y mantener.
 - Los archivos JavaScript almacenados en caché pueden acelerar la carga de las páginas

```
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

```
<script src="myScript.js"></script>
```

Archivo externo: myScript.js

```
function myFunction() {
    document.getElementById("demo").innerHTML = "Paragraph changed.";
}
```


3.1 Introducción al Lenguaje

- JavaScript puede "mostrar" datos de diferentes maneras:
 - Escribiendo en un elemento HTML, usando *innerHTML* o *innerText*.
 - Escribiendo en la salida HTML usando *document.write()*.
 - Solo para pruebas
 - Escribiendo en un cuadro de alerta, utilizando *window.alert()*.
 - Escribiendo en la consola del navegador, usando *console.log()*.

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My First Paragraph</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "<h2>Hello World</h2>";
</script>

</body>
</html>
```


3.2 Manejo de Frameworks

- **Librería:** es un conjunto de código reutilizable que resuelve un problema específico y que una aplicación llama para usar sus funciones
 - React
 - jQuery
- **Framework:** es una estructura de aplicación completa que contiene múltiples librerías y establece cómo la aplicación se escribe y se organiza, controlando el flujo general de la ejecución
 - Vue
 - Angular



3.2 Manejo de Frameworks

- Librería jQuery:
 - jQuery es una librería de JavaScript liviana que permite "escribir menos, hacer más".
 - jQuery simplifica enormemente la programación en JavaScript.
 - Manipulación de HTML/DOM
 - Manipulación de CSS
 - Métodos de eventos HTML
 - Efectos y animaciones
 - AJAX
 - Utilerías



3.2 Manejo de Frameworks

- Librería jQuery:
 - Versión de producción
 - Versión de desarrollo
 - La sintaxis básica es: `$( selector ). action ()`
 - Un signo \$ para definir/acceder a jQuery
 - Un *selector* para "consultar (o encontrar)" elementos HTML
 - *Un action* jQuery () que se realizará en el elemento(s)
- Ejemplos:
 - `$(this).hide()` - oculta el elemento actual.
 - `$("p").hide()` - oculta todos los elementos `<p>`.
 - `$(".test").hide()` - oculta todos los elementos con `class="test"`.
 - `$("#test").hide()` - oculta el elemento con `id="test"`.

```
<head>
<script src="jquery-3.7.1.min.js"></script>
</head>
```

Google CDN:

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
</head>
```

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/
3.7.1/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#test").hide();
  });
});
</script>
</head>
<body>

<h2>This is a heading</h2>

<p>This is a paragraph.</p>
<p id="test">This is another paragraph.</p>

<button>Click me</button>

</body>
</html>
```

This is a heading

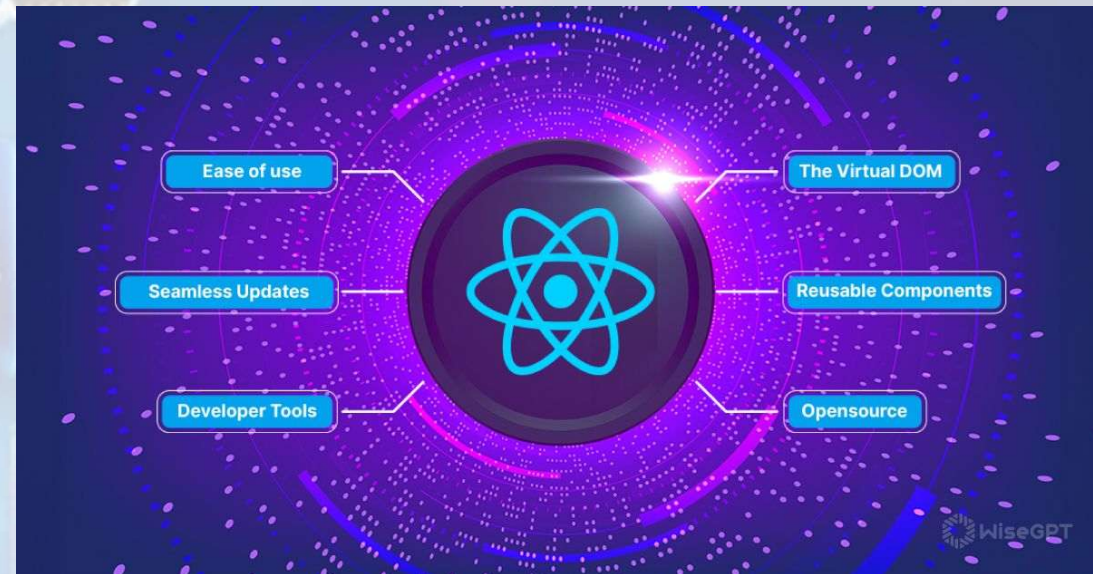
This is a paragraph.

This is another paragraph.

Click me

3.2 Manejo de Frameworks

- Librería React:
 - Librería de JavaScript para crear interfaces de usuario (Front-End).
 - Se utiliza para crear aplicaciones de una sola página.
 - Permite crear componentes de UI reutilizables.
 - React fue desarrollado por el ingeniero de software de Facebook Jordan Walke.
 - React también se conoce como React.js o ReactJS.



3.2 Manejo de Frameworks

- Librería React:
 - Crea un DOM VIRTUAL en memoria.
 - Sólo cambia lo que necesita ser cambiado
- Preparación:
 - Instalar Node.js
 - Instalar herramienta de compilación (Vite)
 - Crear aplicación React
 - Instalar dependencias
 - Ejecutar aplicación

```
node -v
```

```
npm install -g create-vite
```

```
npm create vite@latest my-react-app -- --template react
```

```
npm install
```

```
npm run dev
```

```
VITE v6.3.5 ready in 217 ms  
  
→ Local: http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

```
main.jsx  index.html  
  
import { createRoot } from 'react-dom/client';  
  
function Hello() {  
  return (  
    <h1>Hello World!</h1>  
  );  
}  
  
createRoot(document.getElementById('root')).render(  
  <Hello />  
);
```

```
<!doctype html>  
<html lang="en">  
  <body>  
    <div id="root"></div>  
    <script type="module" src="/src/main.jsx"></script>  
  </body>  
</html>
```

localhost:5173

Hello World!

3.2 Manejo de Frameworks

- Vue:
 - Las interfaces de usuario integradas en Vue se actualizan automáticamente cuando cambian los datos.
 - Vue es fácil de aprender.
 - Evan You, un empleado de Google que utiliza AngularJS, comenzó a desarrollar Vue en 2013.
 - La versión 1.0 de Vue se lanzó en 2015.



3.2 Manejo de Frameworks

- Vue:
 - Paso 1: Página HTML
 - Paso 2: Agregar un <div>
 - Paso 3: Agregar un enlace a Vue
 - Paso 4: Agregar la instancia de Vue
 - Paso 5: Mostrar 'mensaje' con interpolación de texto

```
<!DOCTYPE html>
<html>
<head>
  <title>My first Vue page</title>
  <style>
    #app {
      display: inline-block;
      padding: 10px;
      font-size: x-large;
      background-color: lightgreen;
    }
  </style>
</head>
<body>

  <h1>Vue Example</h1>

  <p>The message is taken from 'data' inside the Vue instance
  by writing {{ message }} inside the div with id="app".</p>

  <div id="app">
    {{ message }}
  </div>

  <script src="https://unpkg.com/vue@3/dist/vue.global.js">
  </script>

  <script>
    const app = Vue.createApp({
      data() {
        return {
          message: "Hello World!"
        }
      }
    })

    app.mount('#app')
  </script>

</body>
</html>
```

Vue Example

The message is taken from 'data' inside the Vue instance by writing {{ message }} inside the div with id="app".

Hello World!

3.2 Manejo de Frameworks

- Angular:
 - Permite crear aplicaciones escalables de una sola página con TypeScript
 - Creado con TypeScript
 - Utiliza componentes y plantillas
 - Incluye enrutamiento, formularios, HTTP e inyección de dependencia.
 - Admite API independientes (no se necesitan NgModules)
 - Funciona muy bien para aplicaciones de una sola página (SPA)
 - Declarar la interfaz de usuario con plantillas basadas en HTML
 - Vincular datos con interpolación y directivas
 - Manejar eventos y entradas de usuario
 - Obtener datos con HttpClient (Observables)



3.2 Manejo de Frameworks

- Angular:

- Preparación:

- Paso 1: tener Node.js
 - Paso 2: instalar Angular CLI
 - Paso 3: crear aplicación
 - Paso 4: ejecutar aplicación

- Aplicación:

- Paso 1: Abrir *src/main.ts*
 - Paso 2: *Element host* en *index.html*
 - Paso 3: Vincular algunos datos

- Estructura:

- *src/main.ts*- Inicia la aplicación con *bootstrapApplication*.
 - *src/app/app.component.ts* Componente raíz (si se utiliza). También se puede definir la raíz en línea *main.ts*.
 - *src/app/-* Donde se agregan los componentes y características.
 - *src/index.html*- Página de host que contiene *<app-root>*.
 - *src/styles.css*- Estilos globales para la aplicación.
 - *angular.json*- Configuración del espacio de trabajo angular (build, serve, test).
 - *package.json*- Scripts y dependencias.

```
ng new my-angular-app
```

```
cd my-angular-app  
ng serve
```

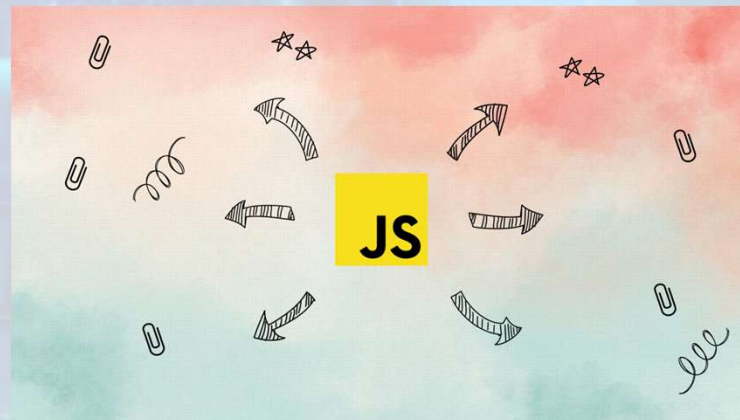
```
main.ts  index.html  
  
import { bootstrapApplication } from '@angular/platform-browser';  
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-root',  
  standalone: true,  
  template: `<h1>Hello, {{ name }}!</h1>`  
})  
class App { name = 'Angular 20'; }  
  
bootstrapApplication(App);
```

Hello, Angular 20!

```
<!doctype html>  
<html lang="en">  
<head>  
  <meta charset="utf-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1">  
  <title>Angular First App: Binding</title>  
</head>  
<body>  
  <app-root></app-root>  
</body>  
</html>
```

3.3 Estructuras de Control

- El flujo de control es el orden en el que se ejecutan las declaraciones en un programa.
- De forma predeterminada, JavaScript ejecuta el código de arriba a abajo y de izquierda a derecha.
- Las declaraciones de flujo de control permiten cambiar ese orden , en función de condiciones, bucles o palabras clave.



3.3 Estructuras de Control

- Flujo Predeterminado
 - Ejecuta el código secuencialmente (de arriba a abajo / de izquierda a derecha).

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
let x = 5;
let y = 6;
let z = x + y;

document.getElementById("demo").innerHTML = "The
value of z is " + z;
</script>

</body>
</html>
```

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is 11

3.3 Estructuras de Control

- Flujo Condicional

- *if*
- *if...else*
- *switch*
- *ternary* (?)

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript if Statement</h1>
<p id="demo"></p>

<script>
let age = 18;
let text = "Unknown"

if (age >= 18) {
  text = "Adult";
} else {
  text = "Minor";
}

document.getElementById("demo").innerHTML = "The
group is " + text;
</script>
</body>
</html>
```

JavaScript if Statement

The group is Adult

3.3 Estructuras de Control

- Bucles
 - *for*
 - *while*
 - *do...while*

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Loops</h1>
<h2>The for Loop</h2>

<p id="demo"></p>

<script>
let text = "";

for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript Loops

The for Loop

The number is 0
The number is 1
The number is 2
The number is 3
The number is 4

3.3 Estructuras de Control

- Salto

- *break*
- *continue*
- *return*
- *throw*

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript break</h1>
<h2>The break Statement in a Loop</h2>

<p>Break the loop when the loop counter is 3:</p>

<p id="demo"></p>

<script>
let text = "";

for (let i = 0; i < 10; i++) {
  if (i === 3) { break; }
  text += "The number is " + i + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript break

The break Statement in a Loop

Break the loop when the loop counter is 3:

The number is 0
The number is 1
The number is 2

3.3 Estructuras de Control

- Funciones
 - Bloques de código invocables y reutilizables

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>

<p>Call a function to compute the product of p1 and
p2 and return the result:</p>

<p id="demo"></p>

<script>
// Function to compute the product of p1 and p2
function myFunction(p1, p2) {
  return p1 * p2;
}

let result = myFunction(4, 3);
document.getElementById("demo").innerHTML = "The
result is: " + result;
</script>

</body>
</html>
```

JavaScript Functions

Call a function to compute the product of p1 and p2 and return the result:

The result is: 12

3.3 Estructuras de Control

- Flujo Asíncrono
 - *Callbacks, Promises, y async/await* manejan el flujo asincrono (events, network calls):

```
async function fetchData() {  
  let data = await fetch("https://api.example.com/data");  
  // code to execute  
}
```


3.4 Manipulación de Objetos

- Un **objeto** es una variable que puede contener muchas variables.
- Los objetos son colecciones de **pares clave-valor**, donde cada clave (conocida como **nombre de propiedad**) tiene un valor.
- Los objetos pueden describir cualquier cosa, como casas, coches, personas, animales o cualquier otro tema.
- Para declarar objetos es con la palabra clave *const*.
- Cuando un objeto se declara con *const*, no es posible reasignarlo posteriormente para que apunte a una variable diferente.
- Esto no hace que el objeto sea inmutable. Aún se pueden modificar sus propiedades y valores.
- No es necesario utilizar *new Object()*.
- Para facilitar la lectura, simplicidad y velocidad, utilice un **objeto literal** en su lugar.

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>Creating an Object</h2>

<p id="demo"></p>

<script>
// Create an Object:
const car = {type:"Fiat", model:"500", color:"white"};

// Display Data from the Object:
document.getElementById("demo").innerHTML =
"The car type is " + car.type;
</script>

</body>
</html>
```

JavaScript Objects

Creating an Object

The car type is Fiat

```
// Create an Object
const person = new Object({
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
});
```

3.4 Manipulación de Objetos

- Propiedades

```
objectName.propertyName
```

```
objectName["propertyName"]
```

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Object Properties</h1>
<h2>Access a Property with .</h2>

<p id="demo"></p>

<script>
const person = {
  firstname: "John",
  lastname: "Doe",
  age: 50,
  eyecolor: "blue"
};

document.getElementById("demo").innerHTML =
person.firstname + " is " + person["age"] + " years
old.";
</script>

</body>
</html>
```

JavaScript Object Properties

Access a Property with .

John is 50 years old.

3.4 Manipulación de Objetos

- Métodos
 - Los métodos de objeto son acciones que se pueden realizar sobre objetos.
 - Los métodos de objeto son definiciones de funciones almacenadas como valores de propiedad

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>Object Methods</h2>
<p>A method is a function definition stored as a property value.</p>

<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>
</body>
</html>
```

JavaScript Objects

Object Methods

A method is a function definition stored as a property value.

John Doe

3.4 Manipulación de Objetos

- Visualización

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>Displaying Properties</h2>

<p id="demo"></p>

<script>
// Create an Object
const person = {
  name: "John",
  age: 30,
  city: "New York"
};

// Display Object
let text = person;

// Display Properties
let text2 = person.name + ", " + person.age + ", " + person.city;

document.getElementById("demo").innerHTML = text + "<br>" + text2;
</script>

</body>
</html>
```

JavaScript Objects

Displaying Properties

[object Object]
John, 30, New York

3.4 Manipulación de Objetos

- Constructores:
 - Para crear un tipo de objeto se utiliza la función constructora de objetos .
 - Se considera una buena práctica nombrar las funciones constructoras con la primera letra mayúscula.

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Objects</h1>
<h2>Object Constructors</h2>

<p id="demo"></p>

<script>
// Constructor Function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}

// Create a Person object
const myFather = new Person("John", "Doe", 50, "blue");

// Display age
document.getElementById("demo").innerHTML =
"My father is " + myFather.age + ".";
</script>

</body>
</html>
```

JavaScript Objects

Object Constructors

My father is 50.

3.4 Manipulación de Objetos

- Los objetos son contenedores de propiedades y métodos.
- Las propiedades se denominan Valores.
- Los métodos son funciones almacenadas como propiedades.
- Las propiedades pueden ser valores primitivos, funciones o incluso otros objetos.
- Los constructores son prototipos de objetos.
- En JavaScript, casi "todo" es un objeto.
 - Los objetos son objetos
 - Las matemáticas son objetos
 - Las funciones son objetos
 - Las fechas son objetos
 - Las matrices son objetos
 - Los mapas son objetos
 - Los conjuntos son objetos
- Todos los valores de JavaScript, excepto los primitivos, son objetos.

Tipo	Valor de ejemplo
string	"Hola"
number	3.14
boolean	verdadero
bigint	12345678901234
null	nulo
undefined	indefinido
symbol	símbolo